

Berliner Hochschule für Technik

Fachbereich VII: Elektrotechnik - Mechatronik - Optometrie

Bachelorarbeit

Vergleich verschiedener Dateisysteme für den SD-Karten-Einsatz in eingebetteten Systemen

Autor: Michael Kolorz

Matrikelnummer: 100359

Studiengang: Elektrotechnik (B.Eng.)

Schwerpunkt: Kommunikationstechnik

Erstprüfer: Prof. Dr. Peter Gober

Zweitprüfer: Prof. Dr. David Dietrich

Semester: Sommersemester 2026

Abgabedatum: 1. Oktober 2026

Inhaltsverzeichnis

1	Einleitung	2
2	SPI-Treiber und SD-Karten	2
3	Dateisysteme	3
3.1	Einleitung	3
3.2	FatFS	3
3.2.1	Einleitung	3
3.2.2	Implementierung	4
	Quellenverzeichnis	5

1 Einleitung

Blah blub

2 SPI-Treiber und SD-Karten

Tabelle 1: Pin-Belegung einer Micro-SD-Karte im SPI-Modus

Pin	SD-Modus	SPI-Modus	Beschreibung (SPI)	STM32 G431 Pin
1	DAT2	<i>Nicht genutzt</i>	Frei lassen (oder Pull-up)	–
2	DAT3 / CD	CS	Chip Select (Low-aktiv)	GPIO (z. B. PA4)
3	CMD	DI	Data In (Dateneingang)	PA7 (MOSI)
4	VDD	VDD	Stromversorgung (3,3 V)	3,3 V
5	CLK	SCLK	Taktleitung	PA5 (SCK)
6	VSS	VSS	Masse	GND
7	DAT0	DO	Data Out (Datenausgang)	PA6 (MISO)
8	DAT1	<i>Nicht genutzt</i>	Frei lassen (oder Pull-up)	–

Tabelle 2: Pin-Belegung einer Standard-SD-Karte im SPI-Modus

Pin	SD-Modus	SPI-Modus	Beschreibung (SPI)	STM32 G431 Pin
1	DAT3 / CD	CS	Chip Select (Low-aktiv)	GPIO (z. B. PA4)
2	CMD	DI	Data In (Dateneingang)	PA7 (MOSI)
3	VSS1	VSS	Masse	GND
4	VDD	VDD	Stromversorgung (3,3 V)	3,3 V
5	CLK	SCLK	Taktleitung	PA5 (SCK)
6	VSS2	VSS	Masse	GND
7	DAT0	DO	Data Out (Datenausgang)	PA6 (MISO)
8	DAT1	<i>Nicht genutzt</i>	Frei lassen	–
9	DAT2	<i>Nicht genutzt</i>	Frei lassen	–

Tabelle 3: Anschlussbelegung der MicroSD-Karte am STM32G431 im SPI-Modus

MicroSD-Pin	Signalname	Verbindung am STM32G431 / Platine
Pin 1	DAT2	3.3V über internen Pull-up
Pin 2	CS	PA9
Pin 3	DI (MOSI)	PA7
Pin 4	VDD	3.3V Hauptversorgung
Pin 5	SCK	PA5
Pin 6	VSS	GND
Pin 7	DO (MISO)	PA6
Pin 8	DAT1	3.3V über internen Pull-up

3 Dateisysteme

3.1 Einleitung

Eine umfassende Einführung in die Grundlagen von Dateisystemen wird von Tanenbaum und Bos in ihrem Standardwerk *Modern Operating Systems* bereitgestellt [1]. Laut den Autoren werden drei essenzielle Anforderungen an die Langzeitspeicherung von Daten gestellt:

1. Es muss möglich sein, eine sehr große Menge an Informationen zu speichern.
2. Die Informationen müssen das Beenden des Prozesses, der sie verwendet, überdauern.
3. Mehrere Prozesse müssen gleichzeitig auf die Informationen zugreifen können.

Im Prinzip könnte das Problem eine große Menge an Information zu speichern bereits mit zwei Operationen lösen:

1. Einen wählbaren Block lesen
2. Einen wählbaren Block schreiben

Das sind exakt die Operationen, die in dem vorherigen Kapitel 2 für die SD-Karte implementiert wurden. In der Praxis sind diese zwei Operationen allerdings zu primitiv, um Daten effektiv zu verwalten. Tanenbaum und Bos stellen beispielhaft folgende Fragen:

- Wie findet man eine bestimmte Information
- Wie verhindert man (auf Mehrbenutzersystemen), dass ein Benutzer nicht die Daten eines anderen ausliest?
- Woher weiß man, welche Blöcke frei sind?

Für diese und viele andere praktische Probleme hilft uns das Konzept der Datei. Die Datei ist etymologisch ein Neologismus aus Daten und Kartei. Im Kontext moderner Betriebssysteme definieren Tanenbaum und Bos[1] Dateien als logische Einheiten von Information, die von Prozessen erzeugt werden und diese überdauern. Eine Datei soll erst gelöscht werden, wenn ein Besitzer dies explizit veranlasst.

Wie Dateien strukturiert, benannt, aufgerufen, genutzt, geschützt, implementiert und verwaltet werden ist Sache des Dateisystems. Aus Sicht des Nutzers ist der wichtigste Aspekt eines Dateisystems, wie es sich darstellt: Was eine Datei ausmacht, wie Dateien benannt und geschützt werden, welche Operationen auf Dateien erlaubt sind. Die Details darüber, ob verkettete Listen oder Bitmaps verwendet werden, um den freien Speicherplatz zu verwalten, oder wie viele Sektoren ein logischer Festplattenblock enthält, sind für den Nutzer von keinem Interesse, obwohl sie für die Entwickler des Dateisystems von großer Bedeutung sind[1]. Während Tanenbaum und Bos den Schwerpunkt auf Dateisystemen für Betriebssysteme legen, was sicherlich der übliche Anwendungsbereich ist, geht es in dieser Arbeit um die Verwendung von Dateisystemen in eingebetteten Systemen, bzw. konkret auf einem STM32-G431 und deren Vor- und Nachteile.

3.2 FatFS

3.2.1 Einleitung

FatFs ist ein plattformunabhängiges, quelloffenes Software-Modul zur Implementierung von FAT/exFAT-Dateisystemen in ressourcenbeschränkten, eingebetteten Systemen. Es wurde von dem Entwickler ChaN[4] entworfen und zeichnet sich durch einen extrem geringen Speicherbedarf sowohl im Programm- als auch im Arbeitsspeicher aus. Das Modul ist vollständig in ANSI C (C89) geschrieben. Dadurch ist es hardwareunabhängig auf praktisch jedem gängigen Mikrocontroller (z. B. ARM, AVR, PIC oder STM32) ohne

Änderungen des Codes direkt lauffähig.

Die Architektur trennt das Dateisystem strikt in zwei logische Schichten: Das Application Interface stellt der Anwendung abstrakte, bekannte Funktionen zur Datei- und Verzeichnisverwaltung wie `f_open`, `f_read`, `f_write` und `f_close` zur Verfügung, während das Media Access Interface (Disk I/O) die Hardware vollkommen vom Dateisystem entkoppelt.

3.2.2 Implementierung

```
1  DSTATUS disk_initialize(BYTE pdrv);
2  DSTATUS disk_status(BYTE pdrv);
3  DRESULT disk_read(...);
4  DRESULT disk_write(...);
5  DRESULT disk_ioctl(...);
```

Quellcode 1: Funktionen, die in der `diskio.cpp` implementiert werden mussten

`ffconf.h`

Die Datei `ffconf.h` ist die zentrale Konfigurationsdatei des FatFs-Moduls. Über den C-Präprozessor lässt sich der Funktionsumfang auf die Hardware-Ressourcen des Mikrocontrollers zuschneiden.

- **Anpassbarer Funktionsumfang:**

- `FF_FS_READONLY`: Schaltet das Schreiben komplett ab, was Flash-Speicher einspart.
- `FF_FS_MINIMIZE`: Entfernt gezielt ungenutzte APIs wie z.B. die Verzeichnisoperationen (`f_mkdir`, `f_opendir`) oder Suchfunktionen.
- `FF_USE_MKFS`: Aktiviert oder deaktiviert die Formatierungsfunktion `f_mkfs`.
- `FF_USE_FASTSEEK`: Beschleunigt den wahlfreien Zugriff auf große Dateien.
- `FF_USE_FIND`: Aktiviert Suchfunktionen wie `f_findfirst()` und `f_findnext()`.

- **Dateinamen und Zeichenkodierung:**

- `FF_USE_LFN`: Schaltet die Unterstützung für lange Dateinamen frei.
- `FF_CODE_PAGE`: Legt die Codepage fest, um Sonderzeichen korrekt darzustellen.
- `FF_LFN_UNICODE`: Aktiviert Unicode-Unterstützung für Dateinamen.

- **Speicher- und Laufwerksoptionen:**

- `FF_VOLUMES`: Definiert die Anzahl der logischen Laufwerke, die gleichzeitig verwaltet werden können.
- `FF_MAX_SS` & `FF_MIN_SS`: Definiert die vom Dateisystem unterstützten logischen Sektorgrößen. Üblicherweise besitzen SD-Karten eine logische Sektorgröße von 512 Byte, FatFs kann jedoch auch größere Sektorgrößen unterstützen.
- `FF_FS_LOCK`: Legt fest, wie viele Dateien gleichzeitig vor konkurrierenden Zugriffen geschützt werden können.

- **Systemeinstellungen:**

- `FF_FS_TINY`: Reduziert den RAM-Bedarf pro Datei, indem interne Puffer gemeinsam genutzt werden.
- `FF_FS_EXFAT`: Aktiviert die Unterstützung des exFAT-Dateisystems für SDXC-Speicherkarten.
- `FF_FS_NORTC`: Verwendet feste Zeitstempel, wenn keine Echtzeituhr (RTC) vorhanden ist.
- `FF_FS_REENTRANT`: Aktiviert die Thread-Sicherheit des Dateisystems für Mehrthread-Anwendungen.

Darüber hinaus stellt FatFs zahlreiche weitere Konfigurationsoptionen bereit. Dazu gehören beispielsweise Einstellungen für mehrere Partitionen (`FF_MULTI_PARTITION`), Laufwerksbezeichnungen (`FF_STR_VOLUME_ID`), Dateiattribute (`FF_USE_CHMOD`), Laufwerkslabels (`FF_USE_LABEL`) oder die Unterstützung von TRIM-Befehlen für Flash-Speicher (`FF_USE_TRIM`). Dadurch lässt sich das Dateisystem flexibel an die Anforderungen der jeweiligen Embedded-Anwendung anpassen. Eine vollständige Auflistung findet sich einerseits direkt in der `ffconf.h` mit hilfreichen Kommentaren, andererseits auf der [offiziellen Webseite des Entwicklers](#). Aufgrund der verfügbaren Ressourcen des STM32G431 wurde auf eine weitgehend vollständige FatFs-Konfiguration zurückgegriffen, einschließlich der Unterstützung langer Dateinamen und des exFAT-Dateisystems.

Quellenverzeichnis

- [1] Tanenbaum, A. S. & Bos, H. (2015). *Modern Operating Systems* (4. Aufl.). Vrije Universiteit Amsterdam, The Netherlands: Prentice Hall.
- [2] Ababei, C. (2013). *Lecture 12: SPI and SD cards*. Vorlesungsskript, EE-379 Embedded Systems and Applications, University at Buffalo. Abgerufen von https://www.dejazzer.com/ee379/lecture_notes/lec12_sd_card.pdf (Besucht am 6. Juli 2026).
- [3] SD Group Matsushita Electric Industrial Co., Ltd. (Panasonic) (2006). *SD Specifications Part 1: Physical Layer Simplified Specification, Version 2.00*. Technische Spezifikation, SD Card Association. Abgerufen von https://users.ece.utexas.edu/~valvano/EE345M/SD_Physical_Layer_Spec.pdf (Besucht am 8. Juli 2026).
- [4] ChaN (2024). *FatFs - Generic FAT Filesystem Module*. Software-Dokumentation, Electronic Lives Manufacturing. Abgerufen von <https://elm-chan.org/fsw/ff/> (Besucht am 23. Juli 2026).
- [5] Microchip Technology Inc. (2024). *How to interface a microSD media with Microchip SD/MMC Card Readers*. Support Knowledge Base. Abgerufen von <https://support.microchip.com/s/article/How-to-interface-a-microSD-media-with-Microchip-SD-MMC-Card-Readers> (Besucht am 8. Juli 2026).
- [6] Michael Eiler, *C++20: Building a Thread-Pool With Coroutines* <https://blog.eiler.eu/posts/20210512/>